

INSTRUCTOR DASHBOARD

BLUEPRINT

*Complete build specification for a new self-service Instructor portal,
mined field-by-field from the existing MindNavy LMS Admin System.*

MindNavy LMS • Source: full read of backend routes/controllers/services/prisma
schema + frontend pages/components/api + IMPACT_MAP.md
Generated 2026-08-26

 **CRITICAL PREREQUISITE: No instructor authentication exists anywhere in this codebase today. See Section 0 before estimating any page in Section 2.**

Table of Contents

0. Critical Prerequisite — Instructor Authentication Does Not Exist

1. Overview

2. Every Page in Detail (12 pages)

2.1 Dashboard Overview

2.2 My Profile

2.3 My Courses

2.4 My Live Sessions

2.5 My Students

2.6 My Reviews

2.7 My Certifications

2.8 My Competencies & Skills

2.9 My Earnings

2.10 Messages & Notifications

2.11 My Reports & Analytics

2.12 Account Settings

3. Impact Map (Reflections)

3.1 Admin ↔ Instructor Dashboard Live Sync

3.2 Per-Action Reflection Detail

4. Data Dependencies

5. Endpoints Mapping (Master Table)

6. Navigation Map

7. States & Edge Cases

Appendix A — Consolidated Backend Gap List

Appendix B — Identity Model & Query Key Convention

0. Critical Prerequisite — Instructor Authentication Does Not Exist

Before any page in this document can be built, one foundational fact has to be accepted: today, 100% of authentication in this codebase is AdminUser-only. An instructor — an AppUser row with role = INSTRUCTOR — cannot log into anything, anywhere, in the current system. This was confirmed by reading every route file (41 files) and the only auth middleware in the backend.

0.1 Evidence

- backend/src/middlewares/auth.middleware.js — requireAdminAuth is the ONLY auth middleware in the codebase. It validates a Bearer token exclusively against AdminSession → AdminUser. It never queries AppUser or AppUserSession.
- backend/src/routes/admin.routes.js — POST /login is the only login endpoint that exists anywhere in the backend. No auth/instructor/learner login route exists.
- Every one of the 41 route files (instructors, courses, live-sessions, finance, notifications, reports, etc.) is gated by requireAdminAuth and nothing else — there is no requireInstructorAuth, requireAppUserAuth, or equivalent.
- AppUser.passwordHash is nullable and is only ever written by admin-initiated flows (admin creates a user / admin resets a password) — never by self-service registration.
- AppUserSession is a fully-modeled Prisma table (tokenHash, expiresAt, revokedAt, lastUsedAt) but NOTHING in the backend ever calls prisma.appUserSession.create(...). It was scaffolded for a future login flow and never wired up — every reference to it today is admin-side read/revoke (e.g. "Force Logout" on a user).
- AppUserRole.INSTRUCTOR is used pervasively as a data-classification filter (course ownership, review authorship, report groupings) but never as an authentication gate.

0.2 What must be built before Section 2 can ship

[NEEDS BACKEND] A real login endpoint for AppUser (email + password → session token), wiring up the dormant AppUserSession table or issuing a JWT.

[NEEDS BACKEND] A requireInstructorAuth middleware mirroring requireAdminAuth in shape, resolving req.instructor from the session, and rejecting non-INSTRUCTOR-role AppUsers.

[NEEDS BACKEND] Ownership guards on every write endpoint this document reuses from the admin side. Today, zero controllers in the Learning Management, Instructors, or Finance modules compare the caller against course.instructorId / liveSession.instructorId / instructorId path params — req.admin?.id is used only for audit-log attribution, never as an authorization check. Pointed at instructor auth as-is, any authenticated instructor could edit or delete ANY course, not just their own.

[NEEDS BACKEND] Server-side self-scoping on every "instructorId" field that is currently client-suppliable (Course.instructorId on create, LiveSession.instructorId on create, the :id path param on every admin/instructors/:id/* route). The instructor-facing variant of each endpoint must silently force these to req.instructor.id and ignore/reject a mismatched client value.

[NEEDS BACKEND] Password reset / forgot-password flow for AppUser (currently only AdminUser has one).
MFA also currently exists only for AdminUser (TOTP via otplib) — decide whether instructors get parity.

None of this is a small addendum — it is the foundation every single page in Section 2 sits on. Every endpoint marked [REUSE — needs ownership guard] below assumes this work is done first.

0.3 Identity model (unchanged, and worth internalizing before reading further)

An instructor is NOT a separate table. It is an AppUser row with role = INSTRUCTOR, plus an optional 1:1 InstructorProfile side table for teaching-specific fields (specialization, bio, headline, yearsExperience, websiteUrl, linkedinUrl, revenueShareBps). This means, once auth exists:

**self.id === Course.instructorId === LiveSession.instructorId === InstructorDocument.instructorId ===
InstructorReview.instructorId === InstructorCertification.instructorId ===
CompetencyCertification.userId**

No separate "instructor identity" ever needs reconciling against these tables — the logged-in AppUser.id is the join key everywhere.

Same logic covers credentials, not just data: AppUser.email and AppUser.passwordHash are the ONE credential pair. Whatever an admin sets on /instructors → Add Instructor (or an approved InstructorApplication sets) is exactly what the new instructor-login endpoint must check — see 1.3's "Same account, no sync step required" callout. And because status/verification/suspension all live on this same AppUser row, every one of those admin actions is automatically visible to the instructor session on their very next request — this is expanded in 3.1 Admin ↔ Instructor Dashboard Live Sync.

1. Overview

1.1 What is the Instructor Dashboard

A new, separate self-service portal where an INSTRUCTOR — today only ever a row an admin manages from the existing /instructors admin page — logs in directly and manages their own teaching business: their profile, the courses they author, the live sessions they host, the students enrolled in their courses, their reviews, their teaching certifications, their linked competencies, and (once Finance is fully wired for it) their earnings.

Architecturally, it is a second front-end application (or a role-gated route tree in the existing frontend) sitting on top of largely the SAME backend tables the admin panel already reads and writes — Course, CourseEnrollment, LiveSession, InstructorProfile, InstructorReview, InstructorDocument, InstructorCertification, CompetencyCertification, NotificationLog, AdminMessage. The admin side stays the system of record; the instructor side is a scoped, self-service window onto rows the instructor already owns.

1.2 Who uses it

- INSTRUCTORS only — AppUser rows with role = INSTRUCTOR. Never AdminUsers, never LEARNERS.
- An instructor only ever sees data scoped to themselves: their own courses, their own students (only within their own courses), their own reviews, their own documents, their own sessions, their own earnings.
- Admins keep exclusive control of the approval/moderation decisions that affect the whole platform: course approve/reject/unpublish, review moderation (approve/remove/flag), document/certification verify/reject, instructor suspend/reactivate/verify. None of these are exposed on the instructor side — see the ADMIN-ONLY markers throughout Section 2.

1.3 How instructors log in

Does not exist yet — new work (Section 0). Recommended shape, consistent with the existing AdminUser login pattern already in this codebase (email + password → session token, stored session row, Bearer auth on every subsequent request):

- POST /api/instructor/auth/login {email, password} → issues an AppUserSession row, returns a Bearer token — reject if AppUser.role !== "INSTRUCTOR" or status !== "ACTIVE".
- Separate login PAGE/URL from the admin login (/instructor/login vs the existing admin /login) — different audience, should not share a form.
- Same session-token-in-header convention as the admin app for consistency, so the existing apiClient wrapper pattern can be reused with a different base path / token store.
- An instructor account is created ONLY via the existing admin-side flows — either an admin directly adds one (POST /instructors) or an instructor application is approved (PATCH /instructor-applications/:id/approve). There is still no public "instructor self-registers with a password" flow,

and none is implied by this document — password is set by whichever of those two admin-side flows created the account, then changed by the instructor via a Change Password action described in 2.12.

SAME ACCOUNT, NO SYNC STEP REQUIRED — there is exactly one AppUser row per instructor, full stop. The Instructor Dashboard does not get its own account table, its own signup flow, or a provisioning job that copies/mirrors an admin-created instructor into a second system. The email + password an admin types into the Add Instructor modal on /instructors (or that gets set when an InstructorApplication is approved) IS, byte-for-byte, the credential pair the Instructor Dashboard login screen checks. The moment POST /instructors or PATCH /instructor-applications/:id/approve returns 201/200, that person can open /instructor/login and sign in — no email verification step, no "activate your account" link, no delay. This is only true if the new login endpoint (Section 0) authenticates against the exact same AppUser.email + AppUser.passwordHash the admin flow just wrote — do not build a parallel credential store for the instructor side.

1.4 Navigation structure (sidebar pages, in the order a developer should build them)

#	Page	Route	One-line purpose
1	Dashboard	/instructor/dashboard	KPI overview of the instructor's own teaching activity
2	My Profile	/instructor/profile	Bio/specialization, verification status, documents, teaching certifications
3	My Courses	/instructor/courses	Course list + full course builder for owned courses
4	My Live Sessions	/instructor/live-sessions	Schedule/host Zoom sessions, mark attendance
5	My Students	/instructor/students	Learners enrolled in the instructor's own courses
6	My Reviews	/instructor/reviews	Read-only student reviews left on the instructor's courses
7	My Certifications	/instructor/certifications	Teaching credentials/licences (uploaded documents)
8	My Competencies	/instructor/competencies	Skills linked to owned courses + competency certifications held
9	My Earnings	/instructor/earnings	Payout history and revenue share (mostly [NEEDS BACKEND])
10	Messages	/instructor/messages	Admin messages + in-app notifications
11	My Reports	/instructor/reports	Self-scoped analytics mirroring admin Reports → Instructor Analytics
12	Settings	/instructor/settings	Password, notification preferences, sessions/devices

This mirrors the existing AdminLayout.tsx sidebar convention (13 flat nav items, no grouping, each with a lucide-react icon) — do the same here: one flat list, 12 items, no nested menus.

2. Every Page in Detail

Status legend used throughout this section: [EXISTS — reuse as-is once auth exists] means the exact admin endpoint can be called self-scoped with no new backend logic beyond Section 0's ownership guard. [REUSE — needs ownership guard] means the endpoint exists but currently accepts an arbitrary :id / instructorId and must be hardened to reject anything but the caller's own id. [NEEDS NEW ENDPOINT] means no query shape exists that returns this data scoped to one instructor — new service code is required. [NEEDS BACKEND] means the underlying capability itself does not exist yet (no writer, no model, or a documented deliberate gap). [ADMIN-ONLY] means the action is intentionally NOT exposed to instructors — it stays an admin decision.

2.1 Dashboard Overview — /instructor/dashboard

Landing page after login. Nine self-scoped KPI cards mirroring the admin Dashboard Overview pattern, plus two charts and three widgets, all scoped to the logged-in instructor's own data.

KPI Cards

Card	Data	Endpoint	Updates when
My Published Courses	COUNT Course WHERE instructorId=self AND status=PUBLISHED	GET /api/instructor/dashboard/stats	course.approve / .unpublish / .archive on own course
My Total Students	COUNT DISTINCT CourseEnrollment.userId across own courses	GET /api/instructor/dashboard/stats	learner.enroll / .unenroll into any own course
My Live Sessions (upcoming)	COUNT LiveSession WHERE instructorId=self AND status=UPCOMING	GET /api/instructor/dashboard/stats	liveSession.create / .delete for self
Pending Course Approvals	COUNT Course WHERE instructorId=self AND status=PENDING	GET /api/instructor/dashboard/stats	course.submitForApproval / admin course.approve/.reject on own course
My Avg. Rating	AVG InstructorReview.rating WHERE instructorId=self AND status=APPROVED	GET /api/instructor/dashboard/stats	admin review.moderate (approve/remove/flag) on a review of self
Certificates Issued (my courses)	COUNT Certificate WHERE courseId IN (own course ids) AND revokedAt IS NULL	GET /api/instructor/dashboard/stats	certificate.issue / .revoke / .reissue for a student in own course
My Earnings (this period)	SUM InstructorPayout.amount for current period	GET /api/instructor/earnings/summary	admin payout.calculate / .approve / .complete for self
Verification Status	AppUser.verificationState + InstructorProfile.verifiedAt (read-only)	GET /api/instructor/profile	admin instructor.verify action
Account Status	AppUser.status (active/suspended)	GET /api/instructor/profile	admin instructor.suspend / .reactivate

- **My Published Courses:** [NEEDS NEW ENDPOINT] — no self-scoped stats aggregate exists
- **My Total Students:** [NEEDS NEW ENDPOINT] — no instructorId filter on enrollments today

- **My Live Sessions (upcoming):** [PARTIAL] admin GET /live-sessions already supports ?instructorId= — needs forced self-scope + aggregate
- **Pending Course Approvals:** [PARTIAL] admin GET /courses?instructor= already supports this filter — needs forced self-scope
- **My Avg. Rating:** [REUSE — needs ownership guard] same formula already lives in `instructors.service.getStats().avgRating`
- **Certificates Issued (my courses):** [NEEDS NEW ENDPOINT] — GET /certificates supports one courseId at a time, not "all my courses" in one call
- **My Earnings (this period):** [NEEDS BACKEND] no instructor-scoped payouts endpoint exists at all; see 2.9
- **Verification Status:** [REUSE — needs ownership guard] same fields the admin side panel already reads via GET /instructors/:id
- **Account Status:** [REUSE — needs ownership guard] — see 3. Impact Map for the suspend→course-unpublish cascade this must surface

Buttons

- **Button: Widget: Recent Activity** [NEEDS NEW ENDPOINT] mirrors admin `recentActivities`, self-scoped
 - Click: Static list, not a button — included here for completeness
 - Endpoint: GET /api/instructor/dashboard/activity
 - Sends: —
 - Returns: `course_created` / `session_scheduled` / `certificate_issued` / `admin_action` items for self
 - Screen after: —
- **Button: Widget: Messages from Admin (preview)** [REUSE — needs self-scope enforcement] endpoint exists, currently accepts any `recipientId`
 - Click: Shows latest 3 `AdminMessage` rows, "View All" → My Messages page
 - Endpoint: GET /api/messages?recipientId=self
 - Sends: —
 - Returns: `AdminMessage[]`
 - Screen after: —
- **Button: Widget: Upcoming Live Sessions (preview)** [REUSE — needs ownership guard]
 - Click: Shows next 3 own sessions, "View All" → My Live Sessions page
 - Endpoint: GET /live-sessions?instructorId=self&status=upcoming
 - Sends: —
 - Returns: `LiveSession[]`
 - Screen after: —

Charts

- **Chart: My Enrollments — Last 12 Months (bar chart)** [NEEDS NEW ENDPOINT] — same SQL shape as the admin side panel's 12-month chart (`instructors.service` raw SQL), just needs a self-scoped variant
 - Source: GET /api/instructor/dashboard/enrollment-trend
 - Axes/segments: X = month, Y = new enrollments across own courses
- **Chart: My Courses by Status (donut)** [NEEDS NEW ENDPOINT] — mirrors admin GET /instructors/analytics `coursesByStatus`, self-scoped

→ Source: GET /api/instructor/dashboard/stats

→ Axes/segments: segments = Draft / Pending / Published / Archived, own courses only

Known Gaps for This Page

[NEEDS NEW ENDPOINT] Every single widget on this page requires a brand-new self-scoped backend endpoint — none of the nine admin GET /lm/* Overview endpoints or the GET /instructors/analytics endpoint accept an instructorId filter today. Budget this as real backend work, not a thin wrapper.

2.2 My Profile — /instructor/profile

Self-service version of the admin Instructor Profile page — everything except status/verification/suspension, which stay read-only (admin decisions).

Tabs

Tab	Shows	Endpoint
Overview	Avatar, name, email (read-only), specialization, headline, bio, skills, years experience, website/LinkedIn links, verification badge (read-only), status badge (read-only)	GET /api/instructor/profile
Documents	Administrative paperwork (Identity/Contract/Agreement/Tax/Compliance) — same 5 types as the admin Documents tab	GET /api/instructor/profile/documents
Certifications	Teaching credentials (Teaching/Professional/Academic/Technical/Training) — separate entity from Competency Certifications, see 2.8	GET /api/instructor/profile/certifications

Buttons

- **Button: Edit Profile**
 - Click: Opens Edit Profile modal
 - Endpoint: —
 - Sends: —
 - Returns: —
 - Screen after: —
- **Button: Change Password [NEEDS BACKEND] no self-service password change endpoint exists for AppUser today (only admin-triggered reset)**
 - Click: Opens Change Password modal (current + new + confirm)
 - Endpoint: PATCH /api/instructor/profile/password
 - Sends: {currentPassword, newPassword}
 - Returns: {success:true}
 - Screen after: toast confirmation, no data refetch needed
- **Button: Upload Document**
 - Click: Opens Upload Document modal
 - Endpoint: —
 - Sends: —
 - Returns: —
 - Screen after: —
- **Button: Upload Certification**
 - Click: Opens Upload Certification modal
 - Endpoint: —
 - Sends: —
 - Returns: —
 - Screen after: —

- **Button: Withdraw Document (PENDING only)** [REUSE — needs ownership guard] same endpoint as admin archive, self-scoped + PENDING-only rule is new
 - Click: Confirm → removes a document the instructor uploaded before an admin has acted on it
 - Endpoint: DELETE /api/instructor/profile/documents/:docId
 - Sends: —
 - Returns: {id}
 - Screen after: row leaves the list (soft-archive, same as admin archive)

Modals

- **Modal: Edit Profile** [NEEDS NEW ENDPOINT] self-service PATCH mirroring admin PATCH /instructors/:id, restricted to profile fields only
 - Opens when: "Edit Profile" button
 - Fields: Specialization, Headline, Bio, Years Experience, Skills (chips), Website URL, LinkedIn URL — full-name/email/phone stay admin-owned per the existing admin edit-modal convention (Users module owns those fields)
 - Submit: PATCH /api/instructor/profile body: {specialization?, headline?, bio?, yearsExperience?, skills?, websiteUrl?, linkedinUrl?} — same "send every field unconditionally, even empty string, PATCH omission = no-change" contract the admin edit modal already follows
 - After submit: Overview tab re-fetches and re-renders
- **Modal: Upload Document** [REUSE — needs ownership guard] exact same sign→PUT→confirm flow as admin, self-scoped
 - Opens when: "Upload Document" button
 - Fields: File (pdf/png/jpeg/webp, ≤10MB), Type (Identity/Contract/Agreement/Tax/Compliance)
 - Submit: POST /api/instructor/profile/documents/sign → PUT to storage → POST /api/instructor/profile/documents/confirm body: sign: {fileName, fileType, type} · confirm: {path, fileName, type}
 - After submit: Documents tab list re-fetches, new row status=PENDING
- **Modal: Upload Certification** [REUSE — needs ownership guard]
 - Opens when: "Upload Certification" button
 - Fields: Name, Type (Teaching/Professional/Academic/Technical/Training), Issuer, File (optional)
 - Submit: POST /api/instructor/profile/certifications/sign → PUT → POST /api/instructor/profile/certifications body: {name, issuer, type, path?, fileName?}
 - After submit: Certifications tab list re-fetches, new row status=PENDING

Known Gaps for This Page

[ADMIN-ONLY] Verify/Reject on both Documents and Certifications stay admin-only — no instructor self-verification exists or should exist. The instructor tabs here are upload + read-status only.

[NEEDS BACKEND] Change Password has no backend today for AppUser — only AdminUser has a self-service password flow.

[NEEDS BACKEND] Avatar upload — no endpoint found anywhere in the codebase for AppUser or AdminUser avatar upload; AppUser.avatar is a plain nullable string field with no writer.

2.3 My Courses — /instructor/courses

The instructor-facing course authoring workspace: list + the full 5-step wizard (Basic Info → Builder → Settings → Preview → Submit) already built for admins, reused wholesale with ownership guards and instructor locked to self. Approve/Reject/Unpublish/Restore stay admin-only and are NOT available anywhere on this page.

Course List / Sub-tabs

Sub-tab	Shows	Endpoint
All	Every course owned by self	GET /courses?instructor=self
Draft	status=DRAFT, editable	GET /courses?instructor=self&status=Draft
Pending Approval	status=PENDING, read-only (waiting on admin)	GET /courses?instructor=self&status=Pending
Published	status=PUBLISHED, live to learners	GET /courses?instructor=self&status=Published
Archived	status=ARCHIVED	GET /courses?instructor=self&status=Archived
Rejected	status=DRAFT AND rejectionReason IS NOT NULL — Course has no REJECTED enum value; a "rejected" course IS a Draft with a reason attached	client-side filter of the Draft tab response

[REUSE — needs ownership guard] GET /courses already supports ?instructor=<id> — the instructor-facing route must force this to self.id server-side and reject a client-supplied different value.

Table columns / filters / actions

Columns: Title | Category | Students (enrolledCount) | Status badge | Updated | Actions

Filters: search (debounced) | category select

Row actions: View (Quick View modal, read-only preview) | Edit (opens builder at the course's current step) | Submit for Review (Draft only) | Withdraw/Archive (Draft only) | View Rejection Reason (if rejected)

Buttons

- **Button: Create Course** [REUSE — needs ownership guard + forced instructorId] admin payload accepts an arbitrary instructorId today; instructor-facing variant must ignore any client value
 - Click: Opens the 5-step wizard fresh, instructor field hidden and hard-locked to self
 - Endpoint: POST /courses
 - Sends: {title, subtitle?, description?, category?, tags?, language?, level?, thumbnail?, categoryId?} — instructorId FORCED to self server-side, never taken from the client
 - Returns: 201 → CourseDetail, status=Draft
 - Screen after: Navigates into the Builder step
- **Button: Save Draft** [REUSE — needs ownership guard: course.instructorId === self]
 - Click: Saves Step 1 fields without leaving the page
 - Endpoint: PATCH /courses/:id
 - Sends: subset of create fields

- Returns: CourseDetail
- Screen after: Form stays open, "Saved" toast
- **Button: Submit for Review** [REUSE — needs ownership guard] this is the one workflow endpoint an instructor SHOULD be allowed to call — see 3. Impact Map
 - Click: On the Submit step, sends the finished course to the admin approval queue
 - Endpoint: POST /courses/:id/submit
 - Sends: — (server re-validates: title/description/thumbnail present, ≥1 section, ≥1 lesson, every attached quiz has ≥1 question)
 - Returns: {id, status:"Pending"}
 - Screen after: Course moves to the Pending Approval tab; admin's Instructors side-panel "Pending Course Approvals" section gains this row
- **Button: Archive (Draft/Rejected only)** [REUSE — needs ownership guard] recommend NOT exposing this for Published courses on the instructor side — that removal should stay an admin decision (Unpublish/Archive), to avoid an instructor unilaterally pulling a course learners are mid-enrollment in
 - Click: Confirm dialog → soft-deletes an unwanted draft
 - Endpoint: DELETE /courses/:id
 - Sends: —
 - Returns: {id, status:"Archived"}
 - Screen after: Row leaves the active tabs, appears greyed in Archived
- **Button: Restore (Archived, own-Draft-origin only)** [REUSE — needs ownership guard]
 - Click: Confirm → brings an archived draft back
 - Endpoint: POST /courses/:id/restore
 - Sends: —
 - Returns: {id, status:"Draft"}
 - Screen after: Row reappears in Draft tab

Course Builder (Step 2) — sections & lessons

Reuses backend/src/routes/courseBuilder.routes.js wholesale. Section list with inline-edit title, up/down reorder (one bulk PATCH /courses/:courseId/reorder call per move), delete (confirms lesson count). "+ Add Lesson" opens the Lesson modal: Title, Type (TEXT / VIDEO_URL toggle), content — TEXT = textarea, VIDEO_URL = Paste URL / Upload File (sign→PUT→confirm) / From Library (search Content Library, scoped — see gap below), Duration (minutes).

[REUSE — needs ownership guard] every section/lesson endpoint resolves ownership transitively via section→course.instructorId; the guard must do that join, not just check a direct field.

Course Settings (Step 4)

Pricing (Free/Paid, Price, Currency), Enrollment (Limit, Visibility Public/Private/Unlisted), Features (Issue completion certificates toggle, Enable drip content toggle), Access Rules (Require enrollment, Start/End dates, Allowed Groups multi-select, Prerequisite Courses multi-select), SEO (Title ≤70, Description ≤200).

Endpoint: PATCH /courses/:id/settings — same field set as admin, same "diff and PATCH only what changed" behavior. [REUSE — needs ownership guard]

Course Preview (Step 5) & Quick View

Read-only render of GET /courses/:id/preview (course + full section/lesson tree). Desktop/Mobile CSS toggle. Same component reused for both the wizard's Preview step and the list's Quick View modal.

[REUSE — needs ownership guard]

Quiz Builder (attached to own courses)

Full reuse of backend/src/routes/quizzes.routes.js: quiz list scoped to courseId ∈ (own courses), Create/Edit Quiz (title, description, passing grade, attempts allowed, time limit, randomize toggle), question editor supporting all 6 types (Multiple Choice, True/False, Multi-Select, Essay, Fill in the Blank, Matching), bulk reorder.

[REUSE — needs ownership guard resolving quiz.courseId → course.instructorId] this join-based guard does not exist anywhere in the codebase today — build it once, reuse for every quiz write endpoint.

Content Library (scoped)

Instructors should see: items they uploaded themselves, plus items already linked to one of their own courses via CourseContentUsage. Upload flow (sign→PUT→confirm) reused as-is.

[NEEDS NEW ENDPOINT] Content Library has no owner-scoping today — CourseContent.uploadedBy is an admin id, not necessarily the instructor, and there is no "items visible to instructor X" query. Must be built new.

Certificates for my courses

"Issue Certificate" secondary action available from a course's Students list (see 2.5), scoped to that one course. Template picker reuses the existing CertificateTemplate list (read-only for instructors — template creation/editing stays admin-only).

[REUSE — needs ownership guard on courseId] POST /certificates already takes {userId, courseId, templateId?} — guard must verify courseId belongs to self before allowing the write.

[ADMIN-ONLY] Approve, Reject, and Unpublish are intentionally NOT on this page anywhere. They stay admin-only decisions, unchanged from today. Only Submit (Draft→Pending) and Archive (own Draft/Rejected only) are instructor-reachable.

[NEEDS BACKEND] Every write endpoint on this page currently has ZERO ownership check in the admin codebase — req.admin?.id is used only for audit-log attribution. This is the single largest chunk of new backend work in the whole document: a reusable ownership-guard layer for Course/CourseSection/Lesson/Quiz/Question/LiveSession/CourseContent/Certificate, all keyed transitively back to course.instructorId or liveSession.instructorId.

2.4 My Live Sessions — /instructor/live-sessions

Schedule and host Zoom sessions, mark attendance for the roster of an owned session's course.

Tabs

Tab	Shows	Endpoint
All / Upcoming / Live / Ended	LiveSession rows where instructorId=self, status is 100% server-derived from startTime+durationMin (never client-writable)	GET /live-sessions?instructorId=self&status=

Table / List

Columns: Title/Description | Course | Start (+timezone) | Duration | Status badge | Actions

Filters: status chips (All/Upcoming/Live/Ended)

Actions per row: Join (opens joinUrl) | Start (host) (opens startUrl — host-only, this is now the instructor's own meeting so it is appropriate here, unlike the admin panel where it is someone else's host link) | End Session (LIVE only) | Attendance (LIVE/ENDED only) | Edit | Cancel (delete)

Buttons

- **Button: Schedule Session**
 - Click: Opens Session form
 - Endpoint: —
 - Sends: —
 - Returns: —
 - Screen after: —
- **Button: End Session [REUSE — needs ownership guard] — note: this is DB-only, does NOT call the real Zoom API to end the meeting (existing admin-side gap, carries over unchanged)**
 - Click: Confirm → manually ends early
 - Endpoint: PATCH /live-sessions/:id/end
 - Sends: —
 - Returns: LiveSession (status=ENDED)
 - Screen after: Status badge flips, moves to Ended tab
- **Button: Cancel Session [REUSE — needs ownership guard]**
 - Click: Confirm → deletes the session
 - Endpoint: DELETE /live-sessions/:id
 - Sends: —
 - Returns: {id}
 - Screen after: Row removed; Zoom meeting deleted best-effort; any learning-path item referencing it shows missing:true instead of disappearing silently

Modals

- **Modal: Session Form (create/edit)** [REUSE — needs ownership guard + forced instructorId] the admin payload currently takes an arbitrary instructorId — this is one of the clearest "must force to self" cases in the whole spec
 - Opens when: "Schedule Session" or row "Edit"
 - Fields: Title, Description, Link to course (optional, own courses only), Start time (future, datetime-local), Timezone (IANA), Duration minutes (5-1440), Max participants (optional)
 - Submit: POST /live-sessions (create) · PATCH /live-sessions/:id (edit) body: {title, startTime, durationMin?, timezone?, courseId?, description?, maxParticipants?, provider:"ZOOM"} — instructorId FORCED to self server-side, never client-supplied
 - After submit: Real Zoom meeting created/updated (joinUrl/startUrl returned); list re-fetches
- **Modal: Attendance** [REUSE — needs ownership guard] standalone sessions with no courseId show "no roster" — same as admin
 - Opens when: "Attendance" action (LIVE/ENDED sessions)
 - Fields: Roster = enrolled learners of the session's course, per-learner status select (Present/Late/Absent/Excused)
 - Submit: PATCH /live-sessions/:id/attendance body: {records:[{userId, status, joinedAt?, leftAt?, durationMin?, participationScore?}]} (max 300)
 - After submit: Attendance summary updates; feeds the learner's own Attendance tab (2.5) and any attendance-threshold certificate trigger

Known Gaps for This Page

[NEEDS BACKEND] *LiveSession.instructorId* is currently a free-text field in the create/update payload, not derived from the caller. The instructor-facing endpoint must hard-override it to self and ignore any client value — otherwise any instructor could schedule a session under someone else's name.

[NEEDS BACKEND] "End Session" never actually terminates the live Zoom meeting — a participant could still be in the real call after the UI shows Ended. This is an existing, documented admin-side gap that carries over unchanged.

2.5 My Students — /instructor/students

Learners enrolled in the instructor's own courses only. This page needs the most new backend work of any page besides Earnings — nothing in the current Learners/Enrollments modules can filter by instructor today.

[NEEDS NEW ENDPOINT] *CourseEnrollment* has no *instructorId* column and *learners.service.listLearners* has no *instructorId/courseId* filter at all. The only path to "my students" is: look up *Course* rows where *instructorId=self*, collect their *ids*, then join through *CourseEnrollment*. No endpoint does this today.

Table columns / filters / actions

Columns: Student (avatar+name) | Course (which of my courses) | Progress (bar) | Status (Not Started/In Progress/Completed/Overdue) | Enrolled Date | Actions

Filters: search (name/email) | Course select (own courses only) | Status

Row actions: View (opens side panel) | Reset Progress | Unenroll | Message

Side Panel — Student in My Course

Opens when: clicking a student row.

Shows: student identity, which of the instructor's courses they're in + progress per course, Assessments tab (quiz scores — MUST be filtered to quizzes belonging to the instructor's own courses only, not the student's full history), Attendance tab (MUST be filtered to the instructor's own live sessions only).

[NEEDS BACKEND] the existing admin endpoints (GET /learners/:id/progress, /assessments, /attendance) are learner-scoped and return ALL of that learner's data across every instructor's courses — reusing them as-is on the instructor side would leak other instructors' course data (grades, attendance) to a student's other teacher. Each of these three needs a course/instructor-filtered variant, not a straight reuse.

Buttons

- **Button: Reset Progress** **[REUSE — needs ownership guard on courseId ∈ own courses]**
 - Click: Confirm → resets a learner's progress on one of my courses to 0%
 - Endpoint: POST /learners/:id/progress/:courseId/reset
 - Sends: —
 - Returns: updated enrollment
 - Screen after: Progress bar resets to 0% in the panel and the table
- **Button: Unenroll** **[REUSE — needs ownership guard: enrollment.course.instructorId === self]**
 - Click: Confirm → removes the student from my course
 - Endpoint: DELETE /learners/:id/enrollments/:enrollmentId
 - Sends: —
 - Returns: {id}
 - Screen after: Row leaves the list; does NOT revoke an already-issued certificate (existing rule, carried over)
- **Button: Grade Quiz Attempt** **[NEEDS BACKEND]** grading is currently an admin-only verb (gradedById is stamped from req.admin?.id) — needs an instructor-reachable variant guarded to quiz.course.instructorId === self

- Click: Opens Override Grade modal for one attempt
- Endpoint: PATCH /learners/:id/assessments/:aid/grade
- Sends: {score, feedback?}
- Returns: LearnerAssessment
- Screen after: Score/status update in the Assessments tab

- **Button: Mark Attendance**

- Click: Deep-links to the relevant session's Attendance modal on 2.4
- Endpoint: PATCH /live-sessions/:id/attendance
- Sends: —
- Returns: —
- Screen after: —

[NEEDS BACKEND] Attendance itself is read-only system-wide today — there is no correction/mark-present endpoint for admins OR instructors beyond the bulk PATCH on the session (2.4). A per-student single-row correction from this page does not exist as a concept yet; use the session-level Attendance modal instead.

[NEEDS BACKEND] The runtime backing QuizAttempt has, per the schema's own comment, "no learner-facing half" — no learner ever actually submits a quiz through any app UI today; every row is admin-seeded/graded. An instructor dashboard implying real students taking real quizzes needs that submission flow built, which is out of scope for a reflection of the existing admin system.

2.6 My Reviews — /instructor/reviews

Read-only view of student reviews left on the instructor's courses. No moderation actions — approve/remove/flag stay admin-only, since that is platform content moderation, not instructor self-service.

KPI Cards

Card	Data	Endpoint	Updates when
Average Rating	AVG rating WHERE status=APPROVED	GET /api/instructor/reviews/stats	admin review.moderate on any of self's reviews
Total Reviews	COUNT WHERE status != REMOVED	GET /api/instructor/reviews/stats	same

- **Average Rating:** [REUSE — needs ownership guard] same formula as `instructors.service.getStats().avgRating`

Table / List

Columns: Rating (stars) | Comment | Course | Student | Date | Status badge

Filters: Status (Approved/Pending/Flagged) — Removed reviews are hidden entirely, same as they would be for anyone else

Actions per row: — (view only, no row actions)

Known Gaps for This Page

[NEEDS BACKEND] No endpoint anywhere in the system lets a student *CREATE* a review — rows only exist via direct DB/seed insert today. This page can only ever show existing rows; it cannot cause new reviews to accumulate through the app.

[ADMIN-ONLY] Approve/Remove/Flag stay strictly admin-only. Do not add these actions to the instructor page even as a "respond to review" feature — no reply/response field exists on *InstructorReview* at all.

2.7 My Certifications — /instructor/certifications

Teaching credentials/licences the instructor holds (degrees, teaching certificates, professional training) — a DIFFERENT entity from Competency Certifications (2.8). This is InstructorCertification, an uploaded-document model living in the Instructors module, not the Competencies module.

Table / List

Columns: Name | Type badge | Issuer | Status badge | Actions

Filters: Type (Teaching/Professional/Academic/Technical/Training) | Status

Actions per row: Delete (PENDING only, self clean-up before admin reviews it)

Buttons

- **Button: Upload Certification**
 - Click: Opens Upload modal
 - Endpoint: —
 - Sends: —
 - Returns: —
 - Screen after: —
- **Button: Delete (PENDING rows only)** [REUSE — needs ownership guard + PENDING-only restriction]
existing admin endpoint allows deleting ANY status; instructor variant should restrict to PENDING to avoid an instructor erasing an admin's REJECTED decision
 - Click: Confirm → hard delete, only before an admin has verified/rejected it
 - Endpoint: DELETE /instructors/:id/certifications/:certId
 - Sends: —
 - Returns: {id}
 - Screen after: Row disappears permanently (hard delete — no ARCHIVED status on this model)

Modals

- **Modal: Upload Certification** [REUSE — needs ownership guard]
 - Opens when: "Upload Certification" button
 - Fields: Name (required, ≤200), Type (required, select), Issuer (required, ≤200), File (optional, pdf/png/jpeg/webp)
 - Submit: POST .../certifications/sign → PUT → POST .../certifications body: {name, issuer, type, path?, fileName?}
 - After submit: List re-fetches, new row status=PENDING

Known Gaps for This Page

[ADMIN-ONLY] Verify/Reject stay admin-only, unchanged. Reject on this model has NO reason field at all (unlike Documents) — a single-click status flip, so the instructor UI should not expect a rejection reason to display beyond "Rejected".

[NEEDS BACKEND] Do not confuse this page with 2.8 My Competencies — they are genuinely two different Prisma models, two different endpoint families, two different status enums (this one has no EXPIRED state; Competency Certifications do).

2.8 My Competencies & Skills — /instructor/competencies

Two distinct sub-tabs: skills linked to the courses the instructor teaches, and CompetencyCertification rows (skill-mastery certificates) the instructor personally holds as an AppUser.

Tabs

Tab	Shows	Endpoint
Skills in My Courses	Every Skill with at least one SkillCourseMapping pointing at one of the instructor's own courses	GET /api/instructor/competencies/course-skills
My Certifications	CompetencyCertification rows where userId=self — skill/competency mastery certs, e.g. "Certified in React"	GET /competencies/users/:userId/certifications (self-scoped)

Known Gaps for This Page

[NEEDS NEW ENDPOINT] "Skills in My Courses" has NO reverse-lookup endpoint today. GET /competencies/skills accepts no courseId/instructorId param, and the only course-relationship read direction that exists is skill → its courses (GET /competencies/skills/:id), never courses → their skills. Building this page requires either a new endpoint or an N+1 client-side loop over every skill in the system — not viable at scale. Build the new endpoint.

[REUSE] "My Certifications" reuses GET /competencies/users/:userId/certifications, which already works correctly for any AppUser id including an instructor — it just needs the self-scoping enforcement from Section 0, no new query logic.

[NEEDS BACKEND] SkillCourseMapping.courseId is a plain string with no DB-level FK — a deleted course shows as missing:true rather than disappearing silently. Preserve that behavior in the new endpoint.

2.9 My Earnings — /instructor/earnings

Nearly the entire page is [NEEDS BACKEND]. Real models exist (InstructorPayout, Payment) but zero instructor-scoped read path exists, and the Payment table is architecturally live but functionally empty until a real payment gateway is connected — every amount will genuinely be \$0 even after the backend work below ships. That is a documented, deliberate "truthful zero," not a bug to chase.

KPI Cards

Card	Data	Endpoint	Status
Lifetime Earnings	SUM InstructorPayout.amount WHERE instructorId=self AND status=COMPLETED	GET /api/instructor/earnings/summary	NEEDS NEW ENDPOINT
Pending Payout	SUM WHERE status IN (PENDING,APPROVED)	GET /api/instructor/earnings/summary	NEEDS NEW ENDPOINT
Current Revenue Share	InstructorProfile.revenueShareBps / 100 as %	GET /api/instructor/profile	REUSE — needs ownership guard
Last Payout Date	MAX completedAt WHERE status=COMPLETED	GET /api/instructor/earnings/summary	NEEDS NEW ENDPOINT

Table — Payout History

Columns: Period (start–end) | Gross Revenue (recomputed live, not stored) | Revenue Share % (snapshotted at calc time) | Amount | Status badge (Pending/Approved/Processing/Completed/Failed/Held) | Approved/Completed Date

Filters: Status | Date range

Row actions: View Breakdown (recomputes which courses/payments contributed — see gap below; no action buttons — approve/hold/complete stay admin-only)

[NEEDS NEW ENDPOINT] No endpoint accepts an instructorId filter on payouts today — GET /finance/payouts only takes {status, page, limit}. A dedicated GET /api/instructor/earnings (or a hardened GET /finance/payouts?instructorId=self) must be built.

[NEEDS NEW ENDPOINT] InstructorPayout stores only the final computed amount — no per-course/per-payment breakdown is persisted anywhere. "Why is my payout \$X" requires a new recompute-on-read endpoint that reruns the same gross-revenue-in-window query the admin calculatePayouts() job uses, scoped to this one payout's period+instructor.

[NEEDS BACKEND] Transaction.paymentId is overloaded: for type=PAYOUT rows it actually stores InstructorPayout.id, not Payment.id. A naive instructor "transaction history" view joining Transaction.paymentId → Payment will silently mismatch PAYOUT rows. Handle the type branch explicitly.

[NEEDS BACKEND] Payment table is empty until a real gateway (Stripe/PayPal) is connected — every instructor will see \$0 everywhere on this page at launch. Display it as a real, live-computed zero (per IMPACT_MAP Rule R3 — never fabricate a non-zero number), not as "coming soon."

[NEEDS BACKEND] The admin side's own "View Earnings" button (Instructor side panel) has been hard-disabled since before Finance shipped and was never rewired — do not copy that disabled pattern; this page needs the real endpoint built, not another disabled placeholder.

2.10 Messages & Notifications — /instructor/messages

Two tabs over two different real backend models: one-way admin→instructor messages (*AdminMessage*), and the platform-wide in-app notification feed (*NotificationLog*, *channel=IN_APP*).

Tabs

Tab	Shows	Endpoint
Messages from Admin	AdminMessage rows where receiverUserId=self — subject, body, type (Direct/Warning/Policy Update/Announcement/Feedback), priority, sent date	GET /api/admin/messages?recipientId=self
Notifications	NotificationLog rows where userId=self, channel=IN_APP — system/campaign/automation-driven feed, same table the admin bell panel's badge count reads	GET /api/admin/notifications?userId=self

Buttons

- **Button: Mark all read (Notifications tab)** [REUSE — needs self-scope enforcement] exists today, currently accepts an arbitrary userId or none (marks EVERY user's feed read) — must be forced to self
 - Click: Zeroes the unread badge
 - Endpoint: PATCH /notifications/read-all
 - Sends: {userId: self}
 - Returns: {success:true}
 - Screen after: Badge and list re-fetch
- **Button: Mark read (single notification)** [REUSE — needs ownership guard]
 - Click: Click a row
 - Endpoint: PATCH /notifications/:id/read
 - Sends: —
 - Returns: updated row
 - Screen after: Row dims/badge decrements
- **Button: Mark read (single message)** [NEEDS BACKEND] no mark-as-read endpoint exists for AdminMessage at all — MessageStatus enum (SENT/READ/ARCHIVED) exists on the model but nothing ever writes READ
 - Click: Click an AdminMessage row
 - Endpoint: —
 - Sends: —
 - Returns: —
 - Screen after: —

Known Gaps for This Page

[NEEDS BACKEND] AdminMessage is confirmed strictly one-way, admin→user — there is no reply/compose capability for the AppUser side anywhere in the schema or service layer. This tab is read + mark-read only, permanently, unless a genuine two-way messaging feature is scoped separately.

[NEEDS BACKEND] Both *GET /messages?recipientId=* and *GET /notifications?userId=* are, today, admin-console endpoints — any caller holding a valid *AdminUser* bearer token can pass ANY *userId/recipientId* and read that person's feed. Before exposing either to instructor auth, the instructor-facing route **MUST** force the *id* to self and reject a mismatch.

2.11 My Reports & Analytics — /instructor/reports

Self-scoped mirror of the admin Reports & Analytics → Instructor Analytics tab, plus a per-course breakdown mirroring the Learning Management Overview KPI pattern — both currently 100% platform-wide with no instructor filter.

KPI Cards

Card	Data	Endpoint	Updates when
Course Completion Rate	% of own courses' enrollments COMPLETED	GET /api/instructor/reports/overview	enrollment.statusUpdate on own course
Avg. Rating	same formula as 2.1/2.6	GET /api/instructor/reports/overview	admin review.moderate on self
Live Session Attendance	% average attendance across own sessions	GET /api/instructor/reports/overview	attendance recorded on own session

- **Course Completion Rate:** [NEEDS NEW ENDPOINT] admin GET /reports/instructors has no instructorId filter — performanceComparison returns EVERY instructor in one array today
- **Avg. Rating:** [REUSE — needs ownership guard]
- **Live Session Attendance:** [NEEDS NEW ENDPOINT]

Charts

- **Chart: Performance Trend (line)** [NEEDS NEW ENDPOINT]
 - Source: GET /api/instructor/reports/overview
 - Axes/segments: X = dateRange bucket (week/month/quarter), Y = completion rate
- **Chart: Per-Course Breakdown (table + bar)** [NEEDS NEW ENDPOINT] mirrors admin LM Overview KPIs (GET /lm/*), which are 100% global admin-wide aggregates with no instructor scoping anywhere today
 - Source: GET /api/instructor/reports/courses
 - Axes/segments: one row per own course: enrolled, completed, completion %, avg quiz score

Known Gaps for This Page

[NEEDS NEW ENDPOINT] The admin GET /reports/instructors endpoint has no instructorId query parameter in its route, validator, or service signature — confirmed by reading the full service function, which always queries every AppUser with role=INSTRUCTOR. performanceComparison is a flat top-N array; a caller cannot ask the server to filter to just their own row.

[NEEDS BACKEND] avgRating and liveSessionAttendance are available:false in the admin Reports module whenever no data exists — preserve that honesty pattern here rather than rendering a fabricated 0.

[NEEDS NEW ENDPOINT] None of the 9 admin GET /lm/* Overview endpoints (stats/distribution/progress/top-courses/content-stats/courses/activities/live-sessions/filter-options) support instructor scoping. The per-course breakdown chart needs new aggregate service code, not a thin param addition.

2.12 Account Settings — /instructor/settings

Password, notification preferences, and session/device management for the instructor's own AppUser account.

Tabs

Tab	Shows	Endpoint
Password	Change Password form	PATCH /api/instructor/profile/password
Notification Preferences	Email/Push/SMS/Marketing/Learning Alerts/Security toggles + quiet hours (UTC only — no timezone field exists on the model)	GET/PATCH /notifications/preferences/self
Sessions & Devices	Active AppUserSession rows (device/IP/last used) with a Revoke action per row	GET /api/instructor/sessions
Two-Factor Authentication	Setup/disable TOTP MFA — parity with the existing per-admin MFA feature	POST /api/instructor/mfa/setup verify disable

Known Gaps for This Page

[REUSE — needs self-scope enforcement] Notification Preferences: UserNotificationPreference already keys off userId and is a real, working model/endpoint — this tab is close to a straight reuse once self-scoping is enforced.

[NEEDS BACKEND] Sessions & Devices: AppUserSession is a fully-modeled table but, per Section 0, nothing ever writes to it today because no login flow exists yet. Once login is built (Section 0), a self-service list+revoke endpoint is still new work — the only existing code that reads/revokes AppUserSession is admin-side ("Force Logout" on a user), never a self-service view.

[NEEDS BACKEND] Two-Factor Authentication: TOTP MFA (otplib + qrcode) exists today ONLY for AdminUser (POST /auth/mfa/setup|verify|disable). Nothing analogous exists for AppUser. Decide whether instructor parity is in scope for v1 before committing to this tab.

[NEEDS BACKEND] Change Password has no backend for AppUser today (see 2.2) — only an admin-triggered reset-password flow exists (PATCH /learners/:id/reset-password and its Instructors equivalent), never a self-service current-password-verified change.

3. Impact Map (Reflections)

Follows the exact convention of the existing IMPACT_MAP.md: every instructor action lists its endpoint, its immediate on-screen effect, and every OTHER surface — on the instructor side AND the existing admin side — that must reflect it without a manual reload. Query keys follow the same factory pattern as frontend/src/lib/queryKeys.ts; a new instructorDashboard.* / instructorCourses.* namespace should be added there, not invented ad hoc in components (Rule R5 of IMPACT_MAP.md: "this map is code").

3.1 Admin ↔ Instructor Dashboard Live Sync

Both apps read and write the SAME rows (Section 0.3) — there is no sync job, no queue, no copy of the data anywhere. "Live" here means two different things depending on whether the change is a permission/security fact or a content fact, and the two must not be conflated:

- Security/permission facts (status=SUSPENDED/ARCHIVED, role change) are enforced PER-REQUEST, not pushed. requireInstructorAuth must re-check AppUser.status on every single call, exactly like requireAdminAuth already re-checks admin.status on every admin request today. That means a suspended instructor is locked out starting with their VERY NEXT API call or page navigation — effectively immediate — with zero new infrastructure required. **[REUSE existing per-request auth-check pattern]**
- Content facts (a course got approved, a review got flagged, a document got verified) are NOT pushed either — this codebase has no WebSocket/SSE layer anywhere (confirmed: the only cross-component sync mechanism found is a same-tab DOM CustomEvent bridge, invalidation.ts's dispatchBridgeEvents — it cannot reach a different browser, let alone a different app). An admin approving a course in one browser and an instructor staring at their My Courses tab in another browser will NOT see it update on its own. The instructor sees it on their next fetch — page load, tab refocus, or an explicit refetch-on-focus/polling interval this project does not have yet. **[NEEDS BACKEND if true push is required]**

[NEEDS BACKEND — decision required] If the product requirement is genuinely "the instructor dashboard updates within seconds without the user doing anything" for content facts (not just security facts), that needs new infrastructure — either short-interval polling on the pages that matter most (Dashboard KPIs, My Courses status) or a WebSocket/SSE push channel — neither exists anywhere in this codebase today. Recommend polling (30-60s) on the few admin-decision-driven fields (course status, verification, suspension banner) rather than building real-time infra for a v1.

Every Admin action that must reflect on the Instructor Dashboard

Admin action	What actually changes	Instructor-side reflection
instructor.create (POST /instructors)	New AppUser+InstructorProfile row created with a password	Instructor can log in immediately (1.3) — My Profile is populated on first load
instructorApplication.approve	Same AppUser creation path, via the applications queue	Same — instructor gains dashboard access for the first time the moment this returns
instructor.update (admin edits profile fields)	PATCH /instructors/:id	My Profile shows the new values on next fetch — same row, same source of truth (R4)

instructor.verify	PATCH /instructors/:id/verify	Verification badge on My Profile + Dashboard flips on next fetch
instructor.suspend	PATCH /instructors/:id/suspend — ALSO force-unpublishes the instructor's Published courses	Locked out of new actions starting their next request (per-request enforcement, effectively immediate); My Courses shows previously-Published courses back in Draft with an explanatory banner (see 3.2 below) on next fetch
instructor.reactivate	PATCH /instructors/:id/reactivate	Login restored immediately (next request); suspension banner clears on next fetch
instructor.delete (soft archive)	DELETE /instructors/:id — AppUser.status → ARCHIVED	Locked out starting their next request, same mechanism as suspend — permanently unless restored
instructorDoc.verify / .reject	PATCH .../documents/:docId/verify reject	My Profile → Documents tab row status updates on next fetch
instructorCert.verify / .reject	PATCH .../certifications/:certId/verify reject	My Certifications row status updates on next fetch
review.moderate (approve/remove/flag)	PATCH .../reviews/:reviewId/approve remove flag	My Reviews list updates (row appears/disappears/flags) on next fetch
course.approve / .reject / .unpublish	admin-only workflow endpoints, Section 2.3	My Courses tab + Dashboard "My Published Courses"/"Pending Course Approvals" KPIs update on next fetch
payout.approve / .complete	admin-only, Section 2.9	My Earnings payout row status + Dashboard "My Earnings" KPI update on next fetch
InstructorProfile.revenueShareBps edited	via instructor.update	My Earnings "Current Revenue Share %" updates on next fetch
admin sends AdminMessage	POST /messages {recipientId=this instructor}	Messages page gains the row + unread indicator on next fetch
admin sends a notification/announcement	POST /notifications/send or announcement.send targeting this instructor	Notifications tab gains the row + badge count updates on next fetch

Every Instructor action that must reflect back to Admin

Instructor action	Endpoint	Admin-side reflection
Edits own profile (2.2)	PATCH self profile	Admin /instructors side panel + profile page show new values on next fetch — same AppUser/InstructorProfile row (R4), no separate admin-side copy
Uploads a document/certification (2.2/2.7)	POST sign→confirm	Appears in Admin's Documents/Certifications tab for this instructor as PENDING; ideally feeds the admin Pending Approvals widget too — NOT currently wired for this source in the admin codebase, flag as a product decision rather than assume it is built
Creates/edits a course draft (2.3)	POST/PATCH /courses	Appears in Admin Learning Management → Courses tab (global list) and this instructor's side-panel course list immediately (same table, no copy)
Submits a course for review (2.3)	POST /courses/:id/submit	Admin Dashboard "Pending Approvals" KPI +1, Tasks & Reminders "Review Pending Courses" +1, this instructor's side-panel "Pending Course Approvals" section gains the row
Schedules a live session (2.4)	POST /live-sessions	Admin Learning Management → Live Sessions tab + Calendar & Events widget gain the row (global surfaces, unfiltered)
Marks session attendance (2.4)	PATCH .../attendance	Feeds the SAME SessionAttendance rows the admin Learners module already reads for that student — visible on the admin side without any new wiring, plus may

		trigger a certificate auto-issue the admin can see in Certificates
Grades a quiz attempt (2.5)	once the [NEEDS BACKEND] instructor-grading endpoint exists	Admin Learners module → that student's Assessments tab reflects the new score/status (same QuizAttempt row)
Archives own draft course (2.3)	DELETE /courses/:id	Disappears from Admin's Courses tab / moves to Archived — same table
Reads a message or notification (2.10)	PATCH .../read	Does NOT reflect to any admin UI today — read-state is instructor-local only, matching the existing IMPACT_MAP rule that notification.markRead is feed-state-only and skips the blanket-invalidation default. AdminMessage.status DOES have a READ enum value the write technically sets, but no admin screen currently displays a "read by recipient" indicator — flag before assuming one exists.
Changes own password (2.12)	PATCH .../password	No UI reflection anywhere — writes an audit_log entry only, per the existing B3 rule that every mutation is audited regardless of which side triggered it

3.2 Per-Action Reflection Detail

Full endpoint/immediate-effect/reflections breakdown for the highest-impact actions, in the same format as the existing IMPACT_MAP.md.

ACTION: Instructor creates a course (Save Draft)

ENDPOINT: POST /courses (instructorId forced to self)

IMMEDIATE EFFECT: Course appears in My Courses → Draft tab

REFLECTIONS:

- → My Courses (Draft tab) → new row
- → Dashboard → "My Published Courses" unaffected (still Draft) but course count in any all-courses total widget increments
- → Admin Instructors module → this instructor's coursesCount increments on their profile
- → Admin Learning Management → Courses tab gains the row (global list, unfiltered)

ACTION: Instructor submits a course for review

ENDPOINT: POST /courses/:id/submit

IMMEDIATE EFFECT: Course moves Draft → Pending; course card shows "Waiting for admin review"

REFLECTIONS:

- → My Courses → moves from Draft tab to Pending Approval tab
- → Dashboard → "Pending Course Approvals" KPI +1
- → Admin Instructors module → this instructor's side-panel "Pending Course Approvals" section gains this row (existing admin surface, already reads Course.submittedAt)
- → Admin Dashboard → "Pending Approvals" KPI + widget (existing platform-wide KPI, per IMPACT_MAP §5.4 course.submitForApproval)
- → Admin Tasks & Reminders → "Review Pending Courses" task count +1

ACTION: Admin approves the instructor's course

ENDPOINT: POST /courses/:id/approve (admin-only, unchanged)

IMMEDIATE EFFECT: Course status flips to Published on the admin side

REFLECTIONS:

- → Instructor's My Courses → row moves from Pending to Published on next fetch (no action the instructor took — this is a PUSH reflection, so the instructor page must either poll, refetch-on-focus, or receive a real-time signal to show it promptly)
- → Instructor Dashboard → "My Published Courses" KPI +1, "Pending Course Approvals" -1
- → Instructor Messages/Notifications → a NotificationLog / AdminMessage row should ideally be created so the instructor is told "Your course was approved" (existing admin flow does NOT currently send this — flag as a product decision, not assumed built)

ACTION: Admin rejects the instructor's course

ENDPOINT: POST /courses/:id/reject {reason}

IMMEDIATE EFFECT: Course returns to Draft with rejectionReason set (there is no REJECTED status — a rejected course IS a Draft with a reason attached)

REFLECTIONS:

- → Instructor's My Courses → row reappears in Draft tab AND (client-derived) the "Rejected" filter, showing rejectionReason inline
- → Instructor Dashboard → "Pending Course Approvals" KPI -1
- → Editing and resubmitting clears nothing server-side automatically — resubmission is just another POST /courses/:id/submit call once the instructor has made changes

ACTION: Admin suspends the instructor

ENDPOINT: PATCH /instructors/:id/suspend (admin-only)

IMMEDIATE EFFECT: AppUser.status → SUSPENDED. The actual current backend behavior ALSO force-unpublishes every one of the instructor's PUBLISHED courses (unpublishInstructorCourses()) — this contradicts the admin dialog's own on-screen copy ("their courses stay published"), which is stale; trust the code, not the dialog text.

REFLECTIONS:

- → Instructor is logged out / session revoked on next request (once instructor auth exists, suspend must also revoke active AppUserSession rows, mirroring the existing admin "force logout on suspend" pattern for other roles)
- → Instructor's My Courses → every previously-Published course silently reappears in the Draft tab — the instructor dashboard MUST surface WHY (e.g. a banner: "Your account was suspended on <date> — your published courses were moved back to Draft"), otherwise this looks like unexplained data loss to the instructor
- → Instructor Dashboard → "My Published Courses" KPI drops to 0 for every unpublished course
- → Admin side: Instructors table status chip, Users table status chip, Active Instructors KPI (all pre-existing, unchanged)

ACTION: Instructor schedules a live session

ENDPOINT: POST /live-sessions (instructorId forced to self)

IMMEDIATE EFFECT: Session appears in My Live Sessions → Upcoming, real Zoom meeting created

REFLECTIONS:

- → My Live Sessions → new row
- → Dashboard → "My Live Sessions (upcoming)" KPI +1
- → If linked to a course → that course's enrolled learners should see it on their learner-facing calendar (existing admin Calendar & Events widget already reads live_sessions platform-wide, unchanged)
- → Admin Learning Management → Live Sessions tab gains the row (global list)

ACTION: Instructor marks session attendance

ENDPOINT: PATCH /live-sessions/:id/attendance

IMMEDIATE EFFECT: Attendance summary updates for that session

REFLECTIONS:

- → My Live Sessions → session row's attendance summary updates
- → My Students → each affected student's Attendance tab gains/updates a record
- → Certificate auto-issue: if the course's certificate rule includes an attendance-threshold trigger and the student now crosses it, a Certificate is auto-issued (existing certificateTriggers.service.js path, unchanged) → reflects into My Certificates-for-my-courses on Dashboard and the student's own Certificates tab

ACTION: Instructor uploads a teaching document/certification

ENDPOINT: POST .../documents/confirm or .../certifications

IMMEDIATE EFFECT: New row appears with status=PENDING

REFLECTIONS:

- → My Profile → Documents/Certifications tab gains the row
- → Admin Instructors module → this instructor's Documents/Certifications tab (admin side) gains the same row — one source of truth, same table
- → Admin Pending Approvals widget → should include this if the product wants document/certification verification to route through the same approvals queue used for course/user approvals (currently NOT wired into that widget — existing admin gap, flag rather than assume)

ACTION: Admin verifies/rejects the instructor's document or certification

ENDPOINT: PATCH .../verify or .../reject (admin-only)

IMMEDIATE EFFECT: Row status flips to VERIFIED or REJECTED

REFLECTIONS:

- → Instructor's My Profile → row status badge updates on next fetch (push reflection, same polling/refetch-on-focus consideration as course approval above)
- → No downstream KPI is fed by document/certification status today (confirmed: neither admin `instructors.service.getStats()` nor `.getAnalytics()` reads these counts) — do not invent a KPI card wired to this without a corresponding backend field

ACTION: Instructor reads a message / notification

ENDPOINT: PATCH `/notifications/:id/read` or `/notifications/read-all`

IMMEDIATE EFFECT: Row/all rows flip to read; unread badge decrements

REFLECTIONS:

- → Messages & Notifications page → row dims, badge count updates
- → Any future instructor-sidebar unread badge (mirroring the existing admin sidebar `"/notifications"` dynamic badge pattern) → decrements
- → No admin-side surface changes — this is instructor-local read-state only, same as the existing `IMPACT_MAP` rule for `notification.markRead` (feed-state-only, skips the §2 blanket-invalidation default)

ACTION: Admin runs payout.calculate / .approve / .complete for this instructor

ENDPOINT: POST `/finance/payouts/calculate` · PATCH `/finance/payouts/:id/approve` · `/complete`
(admin-only)

IMMEDIATE EFFECT: InstructorPayout row created/advances through its status lifecycle

REFLECTIONS:

- → Instructor's My Earnings page → new/updated payout row on next fetch (once the self-scoped endpoint from 2.9 is built)
- → Instructor Dashboard → "My Earnings (this period)" KPI updates
- → Admin Finance → Payouts tab (global list, unchanged) and "Instructor Payouts" KPI (sum across ALL instructors, unchanged)

4. Data Dependencies

For each page: what must already exist for it to render anything meaningful, and which other pages depend on data this page produces.

Page	Needs to exist BEFORE it works	Other pages that depend on it
Dashboard Overview	Instructor account must exist (AppUser role=INSTRUCTOR) and be ACTIVE. Everything else degrades gracefully to zero/empty states with no courses/sessions/students yet.	None — this is a pure aggregator; every other page is upstream of it.
My Profile	Instructor account created (admin add-instructor flow or approved application). InstructorProfile row is optional — a fresh instructor has none until they save one.	My Courses' "Instructor" display name/avatar reads this. Admin Instructors module reads the exact same profile row.
My Courses	At minimum an instructor account. A course needs ≥ 1 section + ≥ 1 lesson + title/description/thumbnail before Submit for Review will succeed (server-enforced checklist, same as the admin course.submit validation).	My Students (needs ≥ 1 published/enrolled course to show anything), My Live Sessions (sessions can optionally link to a course), My Reports (per-course breakdown needs courses to exist), Dashboard KPIs (all course-derived).
My Live Sessions	Zoom integration must be connected at the platform level (System Settings → liveSessionsEnabled feature flag must be ON, or every create call 403s). Course link is optional.	My Students' Attendance tab. Dashboard "My Live Sessions" KPI. Any certificate rule with an attendance trigger.
My Students	Requires ≥ 1 course with ≥ 1 enrollment. Enrollment itself is created by an admin or via the learner-side enroll flow — an instructor cannot self-enroll a student from anywhere in this document (no such action exists in the admin system either).	My Reports' completion-rate figures. Certificate issuance (needs an enrolled, progressing student).
My Reviews	Requires ≥ 1 APPROVED InstructorReview row, which requires a DB-level seed — no in-app path creates one (see 2.6 gap). In practice this page will show empty for every real instructor until that gap is closed.	Dashboard "My Avg. Rating" KPI, My Reports "Avg. Rating" KPI — both read the same formula, so both are empty for the same reason.
My Certifications	None beyond the instructor account — an instructor can upload one immediately after first login.	None downstream.
My Competencies	Requires the Competencies module to have Skill rows with a SkillCourseMapping pointing at one of the instructor's courses (admin-side setup), and/or a CompetencyCertification already issued to this AppUser by an admin.	None downstream.
My Earnings	Requires InstructorProfile.revenueShareBps to be set (admin sets this), ≥ 1 owned course with SUCCESSFUL Payment rows in a calculated period, and an admin having run payout.calculate for that period. Given the Payment table is empty until a gateway exists, this page is realistically empty for every instructor at launch.	Dashboard "My Earnings" KPI reads the same summary.
Messages & Notifications	Requires an admin to have sent at least one AdminMessage or the platform to have logged at	None downstream.

	least one NotificationLog row for this AppUser.	
My Reports	Same course/enrollment/session prerequisites as My Courses/My Students/My Live Sessions — this page is a read-only rollup of all three.	None downstream — terminal page.
Account Settings	Instructor account + (once built) an active AppUserSession for the Sessions & Devices tab to show anything.	None downstream.

5. Endpoints Mapping (Master Table)

Every instructor-facing endpoint referenced in Section 2, in one place. "Status" uses the same legend as Section 2. Request/Response columns are abbreviated — see the relevant page in Section 2 for the full field list.

Page	Action	Method	Endpoint	Request	Response	Status
Dashb oard	View KPIs	GET	/api/instructor/dashboar d/stats	—	9 KPI values	NEW ENDPOINT
Dashb oard	View enrollmen t trend	GET	/api/instructor/dashboar d/enrollment-trend	—	12-month series	NEW ENDPOINT
Dashb oard	View recent activity	GET	/api/instructor/dashboar d/activity	—	activity[]	NEW ENDPOINT
My Profile	View profile	GET	/api/instructor/profile	—	InstructorProfile+Ap pUser fields	NEW ENDPOINT (self-scoped GET /instructors/:id)
My Profile	Edit profile	PATCH	/api/instructor/profile	profile fields only	updated profile	NEW ENDPOINT (self-scoped PATCH /instructors/:id)
My Profile	Change password	PATCH	/api/instructor/profile/pa ssword	{currentPassword, ne wPassword}	{success}	NEEDS BACKEND — no self-service AppUser password change exists
My Profile	List document s	GET	/api/instructor/profile/d ocuments	?type&status	documents[]	REUSE GET /instructors/:id /documents, self-scoped
My Profile	Sign document upload	POST	/api/instructor/profile/d ocuments/sign	{fileName,fileType,ty pe}	{uploadUrl,path}	REUSE, self- scoped
My Profile	Confirm document upload	POST	/api/instructor/profile/d ocuments/confirm	{path,fileName,type}	InstructorDocument	REUSE, self- scoped
My Profile	Withdraw document (PENDING)	DELETE	/api/instructor/profile/d ocuments/:docId	—	{id}	REUSE archive endpoint, self- scoped + PENDING-only
My Profile	List certificati ons	GET	/api/instructor/profile/ce rtifications	?type&status	certifications[]	REUSE GET /instructors/:id /certifications, self-scoped
My Profile	Upload certificati on	POST	/api/instructor/profile/ce rtifications	{name,issuer,type,pa th?}	InstructorCertificati on	REUSE, self- scoped
My Course s	List own courses	GET	/courses?instructor=self &status=	—	courses[]+statusCou nts	REUSE — force self-scope
My Course s	Create course	POST	/courses	title,category,...(instr uctorId forced)	CourseDetail (Draft)	REUSE — force instructorId=se lf
My Course	Edit course	PATCH	/courses/:id	subset of create fields	CourseDetail	REUSE — ownership

s	(Step 1)					guard
My Course s	Archive own draft	DELETE	/courses/:id	—	{id,status:Archived}	REUSE — ownership guard, Draft/Rejected only
My Course s	Restore own archived draft	POST	/courses/:id/restore	—	{id,status:Draft}	REUSE — ownership guard
My Course s	List sections	GET	/courses/:courseId/sections	—	CourseSection[]	REUSE — ownership guard
My Course s	Create/edit/delete section	POST/PATCH/DELETE	/sections/... , /courses/:id/sections	{title,order}	CourseSection	REUSE — ownership guard (via course)
My Course s	Create/edit/delete lesson	POST/PATCH/DELETE	/sections/:id/lessons, /lessons/:id	{title,type,content,durationMin}	Lesson	REUSE — ownership guard (via section→course)
My Course s	Reorder sections/lessons	PATCH	/courses/:courseId/reorder	{sections[],lessons[]}	full tree	REUSE — ownership guard
My Course s	Update settings	PATCH	/courses/:id/settings	pricing/visibility/certificate/etc.	CourseSettings	REUSE — ownership guard
My Course s	Preview course	GET	/courses/:id/preview	—	course+sections	REUSE — ownership guard
My Course s	Submit for review	POST	/courses/:id/submit	—	{id,status:Pending}	REUSE — ownership guard (INSTRUCTOR-REACHABLE, unlike approve/reject/unpublish)
My Course s	List own quizzes	GET	/quizzes?courseId=	—	Quiz[]	REUSE — ownership guard
My Course s	Create/edit/delete quiz	POST/PATCH/DELETE	/quizzes, /quizzes/:id	title,passingGrade,...	Quiz	REUSE — ownership guard (via course)
My Course s	Add/edit/delete question	POST/PATCH/DELETE	/quizzes/:id/questions...	type,prompt,data,points	Question	REUSE — ownership guard
My Course s	Reorder questions	PATCH	/quizzes/:id/reorder	{items[]}	QuizDetail	REUSE — ownership guard
My Course s	Content Library (scoped)	GET	/api/instructor/content-library	?search&type	content[] (own uploads + own-course usages)	NEW ENDPOINT — no ownership exists today
My Course s	Upload content	POST	/content/sign, /content/confirm	{fileName,fileType}→{path,title,courseId}	ContentItem	REUSE — ownership guard on courseId
My Course s	Issue certificate (own course)	POST	/certificates	{userId,courseId,templateId?}	Certificate	REUSE — ownership guard on courseId

My Live Sessions	List own sessions	GET	/live-sessions?instructorId=self	?status	LiveSession[]	REUSE — force self-scope
My Live Sessions	Schedule session	POST	/live-sessions	title,startTime,...(instructorId forced)	LiveSession+joinUrl/startDate	REUSE — force instructorId=self
My Live Sessions	Edit session	PATCH	/live-sessions/:id	subset	LiveSession	REUSE — ownership guard
My Live Sessions	End session	PATCH	/live-sessions/:id/end	—	LiveSession(ENDED)	REUSE — ownership guard (DB-only, real Zoom meeting keeps running)
My Live Sessions	Cancel session	DELETE	/live-sessions/:id	—	{id}	REUSE — ownership guard
My Live Sessions	Mark attendance	PATCH	/live-sessions/:id/attendance	{records[]}	AttendanceRecordResult[]	REUSE — ownership guard
My Students	List students in my courses	GET	/api/instructor/students	?courseId&status&search	students[]	NEW ENDPOINT — no instructorId filter on learners/enrollments exists
My Students	View student progress (my course only)	GET	/api/instructor/students/:id/progress	?courseId	course progress, filtered	NEW ENDPOINT — existing endpoint leaks other instructors' courses
My Students	View student assessments (my course only)	GET	/api/instructor/students/:id/assessments	?courseId	QuizAttempt[], filtered	NEW ENDPOINT — same leak concern
My Students	View student attendance (my sessions only)	GET	/api/instructor/students/:id/attendance	?sessionId	records, filtered	NEW ENDPOINT — same leak concern
My Students	Reset progress	POST	/learners/:id/progress/:courseId/reset	—	enrollment	REUSE — ownership guard on courseId
My Students	Unenroll	DELETE	/learners/:id/enrollments/:enrollmentId	—	{id}	REUSE — ownership guard
My Students	Grade quiz attempt	PATCH	/learners/:id/assessments/:aid/grade	{score,feedback?}	LearnerAssessment	NEEDS BACKEND — admin-only verb today, no ownership-guarded variant

My Reviews	List own reviews	GET	/api/instructor/reviews	?status	InstructorReview[]	REUSE GET /instructors/:id /reviews, self-scope
My Reviews	Review stats	GET	/api/instructor/reviews/stats	—	{avgRating,total}	REUSE avgRating formula, self-scope
My Certifications	List own certifications	GET	/instructors/:id/certifications	?type&status	InstructorCertification[]	REUSE — self-scope
My Certifications	Upload certification	POST	/instructors/:id/certifications/sign, /certifications	{name,issuer,type}	InstructorCertification	REUSE — self-scope
My Certifications	Delete (PENDING only)	DELETE	/instructors/:id/certifications/:certId	—	{id}	REUSE — self-scope + PENDING-only restriction
My Competencies	Skills in my courses	GET	/api/instructor/competencies/course-skills	—	Skill[] with linkedCourse	NEW ENDPOINT — no reverse lookup exists
My Competencies	My competency certifications	GET	/competencies/users/:userId/certifications	—	CompetencyCertification[]	REUSE — self-scope
My Earnings	Earnings summary	GET	/api/instructor/earnings/summary	—	lifetime/pending/latestPayout	NEW ENDPOINT
My Earnings	Payout history	GET	/api/instructor/earnings/payouts	?status&dateRange	InstructorPayout[]	NEW ENDPOINT — no instructorId filter on GET /finance/payouts today
My Earnings	Payout breakdown	GET	/api/instructor/earnings/payouts/:id/breakdown	—	course/payment line items (recomputed)	NEW ENDPOINT — no breakdown persisted anywhere
Messages	List messages from admin	GET	/messages?recipientId=self	—	AdminMessage[]	REUSE — force self-scope
Messages	Mark message read	PATCH	/api/instructor/messages/:id/read	—	{success}	NEEDS BACKEND — no mark-read endpoint exists for AdminMessage
Messages	List notifications	GET	/notifications?userId=self	?read	NotificationLog[]	REUSE — force self-scope
Messages	Mark notification read / all read	PATCH	/notifications/:id/read, /notifications/read-all	{userId:self}	updated	REUSE — force self-scope
My Reports	Overview KPIs	GET	/api/instructor/reports/overview	?dateRange	completionRate/avgRating/attendance	NEW ENDPOINT
My Reports	Per-course breakdown	GET	/api/instructor/reports/courses	—	per-course rows	NEW ENDPOINT

Settings	Notification preferences	GET/PATCH	/notifications/preferences/self	{emailEnabled,...}	UserNotificationPreference	REUSE — force self-scope
Settings	List/revoke sessions	GET/DELETE	/api/instructor/sessions/:id]	—	AppUserSession[]	NEEDS BACKEND — depends entirely on Section 0 login work first
Settings	MFA setup/verify/disable	POST	/api/instructor/mfa/...	TOTP flow	—	NEEDS BACKEND — only AdminUser has this today
Auth	Login	POST	/api/instructor/auth/login	{email,password}	{token}	NEEDS BACKEND — foundational, see Section 0
Auth	Logout	POST	/api/instructor/auth/logout	—	{success}	NEEDS BACKEND

6. Navigation Map

Text tree: Sidebar → page → tab → button/modal → result.

SIDEBAR (flat list, 12 items – mirrors AdminLayout.tsx's un-grouped nav convention)

- 1. Dashboard (/instructor/dashboard)
 - 9 KPI cards (My Published Courses, My Total Students, My Live Sessions, Pending Course Approvals, My Avg. Rating, Certificates Issued, My Earnings, Verification Status, Account Status)
 - Chart: My Enrollments – Last 12 Months
 - Chart: My Courses by Status (donut)
 - Widget: Recent Activity
 - Widget: Messages from Admin (preview) → "View All" → Messages page
 - Widget: Upcoming Live Sessions (preview) → "View All" → My Live Sessions page
- 2. My Profile (/instructor/profile)
 - Tab: Overview
 - Button: Edit Profile → Modal: Edit Profile → PATCH /api/instructor/profile
 - Button: Change Password → Modal: Change Password → PATCH .../password
 - Tab: Documents
 - Button: Upload Document → Modal → sign→PUT→confirm
 - Row action: Withdraw (PENDING only) → DELETE .../documents/:id
 - Tab: Certifications
 - Button: Upload Certification → Modal → sign→PUT→confirm→create
 - Row action: Delete (PENDING only) → DELETE .../certifications/:id
- 3. My Courses (/instructor/courses)
 - Sub-tabs: All / Draft / Pending Approval / Published / Archived / Rejected
 - Button: Create Course → 5-step wizard
 - Step 1 Basic Info → Save Draft → PATCH/POST /courses
 - Step 2 Builder → sections/lessons CRUD + reorder
 - Lesson modal → Paste URL / Upload File / From Library (scoped Content Library)
 - Step 3 (Quiz builder, reached from a lesson or course-level) → Quiz CRUD
 - Question editor (6 types) → reorder
 - Step 4 Settings → PATCH /courses/:id/settings
 - Step 5 Preview → read-only render
 - Step 6 Submit → POST /courses/:id/submit
 - Row action: View → Quick View modal (read-only)
 - Row action: Edit → re-enters wizard at current step
 - Row action: Archive (Draft/Rejected only) → DELETE /courses/:id
 - Row action: Restore (Archived) → POST /courses/:id/restore
- 4. My Live Sessions (/instructor/live-sessions)
 - Status chips: All / Upcoming / Live / Ended
 - Button: Schedule Session → Modal: Session Form → POST /live-sessions
 - Row action: Join → opens joinUrl
 - Row action: Start (host) → opens startUrl
 - Row action: End Session (LIVE) → PATCH .../end
 - Row action: Attendance (LIVE/ENDED) → Modal: Attendance → PATCH .../attendance
 - Row action: Edit → Modal: Session Form → PATCH /live-sessions/:id
 - Row action: Cancel → DELETE /live-sessions/:id
- 5. My Students (/instructor/students)
 - Filters: search | Course select | Status

- └─ Row → Side Panel: Student in My Course
 - └─ Courses tab (progress, own courses only)
 - └─ Button: Reset Progress → POST .../progress/:courseId/reset
 - └─ Button: Unenroll → DELETE .../enrollments/:id
 - └─ Assessments tab (own-course quizzes only)
 - └─ Button: Grade Attempt → Modal → PATCH .../assessments/:id/grade
 - └─ Attendance tab (own sessions only, read-only)
- └─ 6. My Reviews (/instructor/reviews) – read-only, no actions
 - └─ Filters: Status
- └─ 7. My Certifications (/instructor/certifications)
 - └─ Button: Upload Certification → Modal → sign→PUT→confirm→create
 - └─ Row action: Delete (PENDING only)
- └─ 8. My Competencies (/instructor/competencies)
 - └─ Tab: Skills in My Courses (read-only)
 - └─ Tab: My Certifications (read-only, CompetencyCertification)
- └─ 9. My Earnings (/instructor/earnings)
 - └─ KPI cards: Lifetime Earnings / Pending Payout / Revenue Share % / Last Payout Date
 - └─ Table: Payout History → row action: View Breakdown (recomputed, read-only)
- └─ 10. Messages (/instructor/messages)
 - └─ Tab: Messages from Admin (read + mark-read only, no reply)
 - └─ Tab: Notifications
 - └─ Button: Mark all read → PATCH /notifications/read-all
 - └─ Row click → Mark read → PATCH /notifications/:id/read
- └─ 11. My Reports (/instructor/reports)
 - └─ KPI cards: Course Completion Rate / Avg. Rating / Live Session Attendance
 - └─ Chart: Performance Trend (line)
 - └─ Chart/Table: Per-Course Breakdown
- └─ 12. Settings (/instructor/settings)
 - └─ Tab: Password → Change Password
 - └─ Tab: Notification Preferences → toggles → PATCH /notifications/preferences/self
 - └─ Tab: Sessions & Devices → Row action: Revoke
 - └─ Tab: Two-Factor Authentication → Setup / Disable

7. States & Edge Cases

Per page: empty state, loading state, error state, and permission-denied behavior. Follow the existing admin convention throughout: never fabricate a zero or a fake trend arrow (IMPACT_MAP Rule R3) — an unavailable metric renders "—" with a reason string, not a fabricated 0, and an empty list renders a real empty-state message, not a skeleton left spinning forever.

Page	Empty State	Loading State	Error State	Permission Denied
Dashboard	Fresh instructor, 0 courses: all KPIs show 0 (real, computed) except Earnings/Rating which show "—" with a reason ("No reviews yet" / "No payouts yet") — never a fabricated non-zero.	Skeleton cards matching the KpiSkeleton shimmer pattern already used on the admin Dashboard.	Toast + inline retry if any of the aggregate calls fails — the existing admin pattern wraps each KPI query in its own try/catch so one failure never blanks the whole page.	N/A — page itself has no restricted actions; every KPI is inherently self-scoped read
My Profile	A brand-new instructor with no InstructorProfile row shows a "Complete your profile" prompt instead of blank fields (mirrors the admin "No profile" pill).	Standard field skeletons.	Inline field-level error on save failure (e.g. URL fields must start http(s)://, mirrors existing validation).	Editing full name/email/phone is blocked client-side (grey/disabled) — those stay Users-module-owned, same restriction the admin edit modal already enforces.
My Courses	Empty tab shows "You haven't created any courses yet — Create your first course" CTA (Draft/Published/Archived tabs each need their own empty copy, not one generic message).	Table skeleton rows.	Submit-for-review failure shows the FULL errors[] list inline (missing title/thumbnail/no sections/etc.) exactly like the admin Course Submit step — never collapse multiple failures into one generic toast.	Attempting to open a course NOT owned by self (e.g. stale link, guessed URL) → 403, redirect to My Courses with a toast, never a silent 404 that looks like the course was deleted.
My Live Sessions	"No sessions scheduled" empty state with a Schedule CTA.	Table skeleton.	503 amber banner if Zoom is not configured platform-wide; 502 red "transient, try again" banner on a genuine Zoom API failure — reuse the exact two-banner distinction the admin Live Sessions tab already makes.	liveSessionsEnabled feature flag OFF → the whole page (not just the create button) should show a "Live Sessions are currently disabled" state, mirroring the 403 LIVE_SESSIONS_DISABLED the backend already returns.
My Students	"No students enrolled yet" if the instructor has courses but 0 enrollments; distinguish from "You have no courses yet — create one first" if the instructor has 0 courses at all (two different empty states, not one).	Table skeleton.	Standard toast on any failed row action.	Attempting to view a student NOT enrolled in any of self's courses → 403, since the scoped endpoint (Section 5) should never return that student's row in the first place — this is a defense-in-depth check, not a state a well-behaved UI should ever trigger.
My Reviews	"No reviews yet" empty state — expected to be common at launch given no in-app review-creation path exists (2.6 gap).	List skeleton.	Standard toast.	N/A — read-only page, no write actions to gate.
My Certifications	"Add your teaching credentials" empty CTA.	Table skeleton.	Upload failure shows the specific reason (file too large / wrong type / 503	Attempting to delete a VERIFIED or REJECTED row → button hidden

			storage not configured) — reuse the admin Documents/Certifications tab's exact error copy.	entirely (not disabled-with-tooltip) since the restriction is permanent, not situational.
My Competencies	"No skills linked to your courses yet" / "No certifications yet" — two independent empty states per tab.	Tab-level skeleton.	Standard toast.	N/A — both tabs are read-only.
My Earnings	Expect this to show real, live-computed \$0 everywhere at launch (Payment table empty until a gateway exists) — the empty state should read "No payouts yet — earnings begin accruing once a payment gateway is connected," not a broken-looking blank table.	Table skeleton.	Standard toast.	N/A — page is entirely read-only for the instructor; approve/hold/complete stay admin-only.
Messages	"No messages yet" / "You're all caught up" per tab.	List skeleton.	Standard toast.	Attempting to fetch another user's messages/notifications via a tampered request → 403 (Section 0's self-scope enforcement is the actual gate here, not client-side UI).
My Reports	"Not enough data yet" for any chart with zero enrollments/sessions in the selected date range — never render an empty chart canvas with no explanation.	Chart skeleton.	Standard toast; dateRange=custom requires both dateFrom/dateTo or shows inline validation, same as the admin Reports module.	N/A — read-only page.
Settings	N/A — forms always render, no meaningful empty state.	Form skeleton on Sessions & Devices tab while the list loads.	Standard toast; password-change failure shows the specific policy violation (matches the existing admin password-strength validator messaging).	MFA/Sessions tabs should be feature-flagged off entirely (not shown as broken/disabled) until their backend work (Section 0/2.12 gaps) ships.

Appendix A — Consolidated Backend Gap List

Every [NEEDS BACKEND] / [NEEDS NEW ENDPOINT] item in this document, in one place, roughly ordered by how foundational it is. Nothing below row 4 can be built before rows 1-3 exist.

#	Severity	Gap	Referenced in §
1	CRITICAL	Instructor login + session issuance for AppUser (email+password → token). Nothing else in this document works without it.	0
2	CRITICAL	requireInstructorAuth middleware resolving req.instructor from the new session, rejecting non-INSTRUCTOR roles.	0
3	CRITICAL	Ownership guards on every reused admin write endpoint (Course/Section/Lesson/Quiz/Question/LiveSession/CourseContent/Certificate/InstructorDocument/InstructorCertification) — today none of them check the caller against the owning row.	0, 2.3, 2.4
4	HIGH	Server-side forcing of every client-suppliable "instructorId"/":id" field to self (Course create, LiveSession create, every /instructors/:id/* route) — currently a trusting, arbitrary value.	0, 2.3, 2.4
5	HIGH	Self-service password change endpoint for AppUser (only exists for AdminUser today).	2.2, 2.12
6	HIGH	"My Students" instructor-scoped enrollment/learner list — no instructorId filter exists on learners.service or enrollments.service today.	2.5
7	HIGH	Course/instructor-scoped variants of assessments and attendance reads — existing endpoints are learner-scoped only and would leak cross-instructor data if reused as-is.	2.5
8	HIGH	Instructor-scoped Earnings/Payouts read + per-payout breakdown recompute — GET /finance/payouts has no instructorId filter, and InstructorPayout stores no line-item breakdown.	2.9
9	MEDIUM	Full self-scoped Dashboard aggregate endpoints (stats, enrollment trend, activity feed) — none of the 9 admin GET /lm/* endpoints or GET /instructors/analytics support instructor scoping.	2.1
10	MEDIUM	"Skills in My Courses" reverse lookup (courses → skills) — only the opposite direction (skill → its courses) exists today.	2.8
11	MEDIUM	Instructor-scoped Reports (Overview + per-course breakdown) — GET /reports/instructors has no instructorId parameter.	2.11
12	MEDIUM	Content Library owner-scoping ("items visible to instructor X") — no concept of item ownership beyond an admin-attributed uploadedBy field exists today.	2.3
13	MEDIUM	Ownership-guarded quiz-grading endpoint for instructors (today grading is strictly admin-only, gradedById stamped from req.admin).	2.5
14	LOW	Mark-as-read endpoint for AdminMessage (MessageStatus.READ exists on the model, nothing ever writes it).	2.10
15	LOW	Self-service Sessions & Devices list/revoke for AppUser (AppUserSession only has admin-side read/revoke today, and won't exist at all until #1 is built).	2.12
16	LOW	MFA parity for AppUser (TOTP exists only for AdminUser today) — a scope decision, not a bug.	2.12
17	LOW	Avatar upload for AppUser — no endpoint exists for anyone (admin or instructor) today; AppUser.avatar is a writer-less nullable field.	2.2
18	INFO	Two documented UI/behavior discrepancies to resolve, not build: (a) Instructor.rating is hard-null on every list/table surface while stats.avgRating is genuinely live-computed — pick one behavior for the instructor dashboard, don't silently inherit the inconsistency; (b) the admin Suspend dialog's on-screen copy ("courses stay published") is FALSE — the actual code force-unpublishes them — the instructor dashboard must reflect real behavior, not the stale copy.	2.1, 3

Appendix B — Identity Model & Query Key Convention

B.1 Identity model (repeated from Section 0 for reference)

An instructor is an AppUser row with role = INSTRUCTOR, plus an optional 1:1 InstructorProfile side table. self.id is the join key against every one of these existing FK/scalar fields:

Field	Nature	Meaning for self-scoping
Course.instructorId	nullable, unenforced FK, indexed	owns/edits
LiveSession.instructorId	nullable, unenforced FK, indexed	hosts
InstructorDocument.instructorId	FK, cascade delete	administrative paperwork
InstructorReview.instructorId	FK, cascade delete	reviews received (read-only)
InstructorCertification.instructorId	FK, cascade delete	teaching credentials
CompetencyCertification.userId	real Prisma relation	skill-mastery certs held
AdminMessage.receiverUserId	real Prisma relation	messages received
NotificationLog.userId	plain scalar, no relation	notification feed
InstructorPayout.instructorId	plain scalar, no relation	payouts (once endpoint exists)

B.2 Query key convention to add to frontend/src/lib/queryKeys.ts

Follow the exact existing factory pattern (function returning a readonly tuple, grouped by domain namespace). Proposed additions:

```
instructorDashboard: {
  stats:      (): QueryKey => ['instructor-dashboard', 'stats'],
  enrollmentTrend: (): QueryKey => ['instructor-dashboard', 'enrollment-trend'],
  activity:   (): QueryKey => ['instructor-dashboard', 'activity'],
},
instructorSelf: {
  profile:    (): QueryKey => ['instructor-self', 'profile'],
  documents:  (): QueryKey => ['instructor-self', 'documents'],
  certifications: (): QueryKey => ['instructor-self', 'certifications'],
},
instructorCourses: {
  list:      (filters?: Record<string, unknown>): QueryKey =>
    filters ? ['instructor-courses', filters] : ['instructor-courses'],
  detail:    (id: string): QueryKey => ['instructor-courses', id],
  sections:  (courseId: string): QueryKey => ['instructor-courses', courseId, 'sections'],
},
instructorStudents: {
  list:      (filters?): QueryKey => ['instructor-students', filters ?? {}],
  detail:    (id: string): QueryKey => ['instructor-students', id],
},
instructorEarnings: {
  summary:   (): QueryKey => ['instructor-earnings', 'summary'],
  payouts:   (filters?): QueryKey => ['instructor-earnings', 'payouts', filters ?? {}],
},
```

Add matching MutationName members and INVALIDATION_MAP rows in frontend/src/lib/invalidation.ts for every write action in Section 5 — e.g. instructorCourse.submit, instructorProfile.update, instructorSession.schedule, instructorAttendance.mark — mirroring the existing §5-numbered domain-comment convention in that file so a future reader can tell at a glance this is the instructor-side counterpart of an existing admin mutation, not an unrelated one.

Rule to carry over unchanged from IMPACT_MAP.md Rule R4 ("one source of truth per datum"): every number this dashboard shows that ALSO appears somewhere in the admin panel (course counts, student counts, ratings, enrollment trends) must be computed by the SAME service function, just called with a narrower scope — never a second, independently-written query that happens to compute a similar number. Where this document says "REUSE the formula," it means literally call the same function with an instructorId filter added, not reimplement it.